

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 线性方程组解法对比——Jacobi/Gauss-Seidel
迭代法、超松弛法、最速下降法

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/04/09

实验报告提交时间: 2026/04/12

实验目的与要求:

1. 掌握 Jacobi/Gauss-Seidel 迭代法、超松弛法、最速下降法的基本原理和算法步骤
2. 通过编程实现求解线性方程组 $Ax=b$ 的算法, 验证理论结果
3. 比较 Jacobi/Gauss-Seidel 迭代法、超松弛法、最速下降法的优缺点及其在不同初始值条件下的收敛性

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

本实验求解线性方程组 $Ax=b$, 其中:

$$A = \text{diag}(5,5,5,5,5) - (J - I), \quad b = [-1, 1, -2, 2, -3]^T$$

(矩阵 A 对角元素为 5, 非对角元素均为 -1, 为严格对角占优矩阵)

(一) Jacobi 迭代法

将方程组分解为 $A = D - L - U$, 则迭代格式为: $x^{k+1} = D^{-1}(L + U)x^k + D^{-1}b$

每次迭代完全使用上一步的旧值, 各分量可独立计算, 便于并行。收敛条件为谱半径 $\rho(B_J) < 1$, 其中 $B_J = D^{-1}(L+U)$ 。

(二) Gauss-Seidel 迭代法

迭代格式为: $x^{k+1} = (D-L)^{-1}Ux^k + (D-L)^{-1}b$

更新第 i 个分量时立即使用已更新的新值, 收敛速度通常快于 Jacobi 法。对同一矩阵, GS 收敛时谱半径满足 $\rho(B_{GS}) = \rho(B_J)^2$ (对于本题矩阵成立)。

(三) 超松弛法 (SOR)

在 G-S 基础上引入松弛因子 ω ($1 < \omega < 2$), 迭代格式为: $x_i^{k+1} = (1-\omega)x_i^k + \omega \cdot x_i^{(GS)}$

最优松弛因子 $\omega^* = 2 / (1 + \sqrt{1 - \rho^2(B_J)})$, 本题 $\rho(B_J) = 0.8$, 故 $\omega^* \approx 1.25$ 。当 $\omega > \omega_{opt}$ 时, $\rho(B_{SOR}) = \omega - 1$ 。

(四) 最速下降法

适用于对称正定矩阵, 等价于最小化二次泛函 $f(x) = \frac{1}{2}x^T Ax - b^T x$ 。

沿负梯度(残差)方向搜索, 最优步长 $\alpha_k = r^T r / (r^T A r)$, 迭代格式为 $x^{k+1} = x^k + \alpha_k r^k$, 其中 $r^k = b - Ax^k$ 。

收敛速度由条件数 κ 决定: $\rho \leq ((\kappa - 1)/(\kappa + 1))^2$ 。本题 A 的特征值为 1 (1 重) 和 6 (4 重), $\kappa = 6$, 故 $\rho \leq (5/7)^2 \approx 0.510$ 。

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)

求解如下 5×5 线性方程组 $Ax = b$ (结果保留五位有效数字):

$$\begin{bmatrix} 5 & -1 & -1 & -1 & -1 \\ -1 & 5 & -1 & -1 & -1 \\ -1 & -1 & 5 & -1 & -1 \\ -1 & -1 & -1 & 5 & -1 \\ -1 & -1 & -1 & -1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ -2 \\ 2 \\ -3 \end{bmatrix}$$

2. 实验步骤:

(1) 算法实现: 伪代码或算法框图

【Jacobi 迭代法伪代码】

```
Input: A, b, x0, tol, maxIter
x ← x0
for k = 1 to maxIter do
    for i = 1 to n do
        x_new[i] ← (b[i] - Σ_{j≠i} A[i,j] · x[j]) / A[i,i]
    end for
    if ||x_new - x||_∞ < tol then return x_new
    x ← x_new
end for
```

【SOR 迭代法伪代码】

```
Input: A, b, x0, ω, tol, maxIter
x ← x0
for k = 1 to maxIter do
    for i = 1 to n do
        x_GS ← (b[i] - Σ_{j≠i} A[i,j] · x[j]) / A[i,i]
        x[i] ← (1-ω) · x[i] + ω · x_GS
    end for
    if ||Δx||_∞ < tol then return x
end for
```

【最速下降法伪代码】

```
Input: A, b, x0, tol, maxIter
x ← x0; r ← b - A · x
for k = 1 to maxIter do
    if ||r||_2 < tol then return x
```

```

 $\alpha \leftarrow (r^T r) / (r^T A r)$ 
 $x \leftarrow x + \alpha \cdot r$ 
 $r \leftarrow b - A \cdot x$ 
end for

```

(2) 测试验证：使用简单算例验证程序正确性

(3) 参数设置：设置合适的初始值、精度要求等参数

参数	取值	说明
收敛容差 tol	1×10^{-6}	相邻两步 $\ \Delta x \ _{\infty} < \text{tol}$ 则停止
最大迭代次数	1000	防止不收敛时死循环
SOR 松弛因子 ω	1.5	初始设定，后续对 $\omega \in [0.5, 1.9]$ 做参数扫描
初始值 x_0 — 方案 1	$[0, 0, 0, 0]^T$	零向量，最常用默认起点
初始值 x_0 — 方案 2	$[1, 1, 1, 1]^T$	全 1 向量，测试非零起点影响
初始值 x_0 — 方案 3	$[-1, 1, -1, 1]^T$	交替符号向量，测试初始误差方向影响

(4) 运行计算：对目标问题进行求解

(5) 结果记录：记录迭代过程、最终结果和性能指标

精确解 $x^* =$

```

x1 = -0.66667
x2 = -0.33333
x3 = -0.83333
x4 = -0.16667
x5 = -1

```

初始值：零向量 $[0, 0, 0, 0]^T$

```

[Jacobi]      迭代次数:   54  ||x-x*||_inf = 3.51e-06
              解: [-0.66666, -0.33333, -0.83333, -0.16666, -1]
[Gauss-Seidel] 迭代次数:   30  ||x-x*||_inf = 1.74e-06
              解: [-0.66666, -0.33333, -0.83333, -0.16667, -1]
[SOR   $\omega=1.5$ ] 迭代次数:   24  ||x-x*||_inf = 3.22e-07
              解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]
[最速下降法]   迭代次数:   24  ||x-x*||_inf = 2.23e-07
              解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]

```

初始值：全 1 向量 $[1, 1, 1, 1]^T$

```

[Jacobi]      迭代次数:   58  ||x-x*||_inf = 3.83e-06

```

解: [-0.66666, -0.33333, -0.83333, -0.16666, -1]
[Gauss-Seidel] 迭代次数: 33 $\|x-x^*\|_{\text{inf}} = 1.19\text{e-}06$
解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]
[SOR $\omega=1.5$] 迭代次数: 24 $\|x-x^*\|_{\text{inf}} = 3.52\text{e-}07$
解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]
[最速下降法] 迭代次数: 47 $\|x-x^*\|_{\text{inf}} = 2.97\text{e-}07$
解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]

=====

初始值: 交替向量 [-1,1,-1,1,-1]^T

=====

[Jacobi] 迭代次数: 52 $\|x-x^*\|_{\text{inf}} = 3.65\text{e-}06$
解: [-0.66666, -0.33333, -0.83333, -0.16666, -1]

[Gauss-Seidel] 迭代次数: 29 $\|x-x^*\|_{\text{inf}} = 1.80\text{e-}06$
解: [-0.66666, -0.33333, -0.83333, -0.16667, -1]

[SOR $\omega=1.5$] 迭代次数: 25 $\|x-x^*\|_{\text{inf}} = 3.11\text{e-}07$
解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]

[最速下降法] 迭代次数: 11 $\|x-x^*\|_{\text{inf}} = 7.42\text{e-}08$
解: [-0.66667, -0.33333, -0.83333, -0.16667, -1]

=====

SOR 不同松弛因子 ω 对比 (初始值=零向量)

=====

$\omega = 0.5$ 迭代次数 = 86
 $\omega = 0.6$ 迭代次数 = 68
 $\omega = 0.7$ 迭代次数 = 55
 $\omega = 0.8$ 迭代次数 = 45
 $\omega = 0.9$ 迭代次数 = 37
 $\omega = 1.0$ 迭代次数 = 30
 $\omega = 1.1$ 迭代次数 = 24
 $\omega = 1.2$ 迭代次数 = 19
 $\omega = 1.3$ 迭代次数 = 15
 $\omega = 1.4$ 迭代次数 = 19
 $\omega = 1.5$ 迭代次数 = 24
 $\omega = 1.6$ 迭代次数 = 33
 $\omega = 1.7$ 迭代次数 = 46
 $\omega = 1.8$ 迭代次数 = 72
 $\omega = 1.9$ 迭代次数 = 152

最优松弛因子: $\omega^* = 1.3$, 最少迭代次数: 15

3. 程序设计 (具体的 MATLAB 程序)

%% 线性方程组解法对比

```

% Jacobi 迭代法、Gauss-Seidel 迭代法、超松弛法(SOR)、最速下降法
% 结果保留五位有效数字

clear; clc; close all;

%% ===== 定义方程组 =====
A = [ 5 -1 -1 -1 -1;
      -1 5 -1 -1 -1;
      -1 -1 5 -1 -1;
      -1 -1 -1 5 -1;
      -1 -1 -1 -1 5];

b = [-1; 1; -2; 2; -3];

% 精确解（用 MATLAB 内置求解器）
x_exact = A \ b;
fprintf('精确解 x* = \n');
fprintf(' x1 = %.5g\n', x_exact(1));
fprintf(' x2 = %.5g\n', x_exact(2));
fprintf(' x3 = %.5g\n', x_exact(3));
fprintf(' x4 = %.5g\n', x_exact(4));
fprintf(' x5 = %.5g\n\n', x_exact(5));

%% ===== 参数设置 =====
tol = 1e-6; % 收敛容差
maxIter = 1000; % 最大迭代次数
omega = 1.5; % SOR 松弛因子（可调整， $1 < \omega < 2$  为超松弛）

% 三组初始值
x0_list = {zeros(5,1), ones(5,1), [-1;1;-1;1;-1]};
x0_names = {'零向量 [0,0,0,0,0]^T', '全 1 向量 [1,1,1,1,1]^T', '交替向量 [-1,1,-1,1,-1]^T'};

%% ===== 主循环：对每组初始值运行四种方法 =====
for k = 1:length(x0_list)
    x0 = x0_list{k};
    fprintf('===== \n');
    fprintf('初始值: %s\n', x0_names{k});
    fprintf('===== \n');

    % --- 1. Jacobi 迭代法 ---
    [x_jac, iter_jac, res_jac] = jacobi(A, b, x0, tol, maxIter);
    fprintf('[Jacobi] 迭代次数: %4d ||x-x*||_inf = %.2e\n', ...
            iter_jac, norm(x_jac - x_exact, inf));

```

```

fprintf(' 解: [%.5g, %.5g, %.5g, %.5g, %.5g]\n\n', x_jac);

% --- 2. Gauss-Seidel 迭代法 ---
[x_gs, iter_gs, res_gs] = gauss_seidel(A, b, x0, tol, maxIter);
fprintf('[Gauss-Seidel] 迭代次数: %4d ||x-x*||_inf = %.2e\n', ...
iter_gs, norm(x_gs - x_exact, inf));
fprintf(' 解: [%.5g, %.5g, %.5g, %.5g, %.5g]\n\n', x_gs);

% --- 3. SOR 超松弛法 ---
[x_sor, iter_sor, res_sor] = sor(A, b, x0, omega, tol, maxIter);
fprintf('[SOR w=%.1f] 迭代次数: %4d ||x-x*||_inf = %.2e\n', ...
omega, iter_sor, norm(x_sor - x_exact, inf));
fprintf(' 解: [%.5g, %.5g, %.5g, %.5g, %.5g]\n\n', x_sor);

% --- 4. 最速下降法 ---
[x_sd, iter_sd, res_sd] = steepest_descent(A, b, x0, tol, maxIter);
fprintf('[最速下降法] 迭代次数: %4d ||x-x*||_inf = %.2e\n', ...
iter_sd, norm(x_sd - x_exact, inf));
fprintf(' 解: [%.5g, %.5g, %.5g, %.5g, %.5g]\n\n', x_sd);

%% 绘制收敛曲线
figure('Name', sprintf('初始值%d: %s', k, x0_names{k}), ...
'Position', [100+k*50, 100, 700, 450]);
semilogy(res_jac, 'b-o', 'MarkerSize', 3, 'DisplayName', 'Jacobi');
hold on;
semilogy(res_gs, 'r-s', 'MarkerSize', 3, 'DisplayName', 'Gauss-Seidel');
semilogy(res_sor, 'g-^', 'MarkerSize', 3, 'DisplayName',
sprintf('SOR(w=%.1f)', omega));
semilogy(res_sd, 'm-d', 'MarkerSize', 3, 'DisplayName', '最速下降法');
xlabel('迭代次数'); ylabel('残差 ||r_k||_2');
title(sprintf('收敛曲线 - 初始值: %s', x0_names{k}));
legend('Location', 'northeast'); grid on;
hold off;
end

%% ===== SOR 最优松弛因子分析 =====
fprintf('\n=====');
fprintf('SOR 不同松弛因子 $\omega$ 对比 (初始值=零向量)\n');
fprintf('=====');
omega_list = 0.5:0.1:1.9;
iters_omega = zeros(size(omega_list));
x0 = zeros(5,1);
for i = 1:length(omega_list)
[~, it, ~] = sor(A, b, x0, omega_list(i), tol, maxIter);

```

```

iters_omega(i) = it;
fprintf('  $\omega$  = %.1f 迭代次数 = %d\n', omega_list(i), it);
end

figure('Name', 'SOR 松弛因子分析', 'Position', [200, 200, 600, 350]);
plot(omega_list, iters_omega, 'b-o', 'LineWidth', 1.5, 'MarkerSize', 6);
xlabel('松弛因子  $\omega$ '); ylabel('迭代次数');
title('SOR 不同松弛因子下的迭代次数');
grid on;
[~, idx] = min(iters_omega);
fprintf('\n 最优松弛因子:  $\omega^*$  = %.1f, 最少迭代次数: %d\n', omega_list(idx),
iters_omega(idx));

%% =====
%% 子函数定义
%% =====

%% Jacobi 迭代法
function [x, iter, res_hist] = jacobi(A, b, x0, tol, maxIter)
n = length(b);
x = x0;
res_hist = [];
for iter = 1:maxIter
x_new = zeros(n, 1);
for i = 1:n
s = b(i) - A(i, [1:i-1, i+1:n]) * x([1:i-1, i+1:n]);
x_new(i) = s / A(i, i);
end
r = norm(b - A*x_new, 2);
res_hist(end+1) = r; %#ok<AGROW>
if norm(x_new - x, inf) < tol
x = x_new;
return;
end
x = x_new;
end
warning('Jacobi: 达到最大迭代次数, 未收敛');
end

%% Gauss-Seidel 迭代法
function [x, iter, res_hist] = gauss_seidel(A, b, x0, tol, maxIter)
n = length(b);
x = x0;
res_hist = [];

```



```

for iter = 1:maxIter
x_old = x;
for i = 1:n
s = b(i) - A(i, [1:i-1, i+1:n]) * x([1:i-1, i+1:n]);
x(i) = s / A(i, i);
end
r = norm(b - A*x, 2);
res_hist(end+1) = r; %#ok<AGROW>
if norm(x - x_old, inf) < tol
return;
end
end
warning('Gauss-Seidel: 达到最大迭代次数, 未收敛');
end

%% SOR 超松弛迭代法
function [x, iter, res_hist] = sor(A, b, x0, omega, tol, maxIter)
n = length(b);
x = x0;
res_hist = [];
for iter = 1:maxIter
x_old = x;
for i = 1:n
s = b(i) - A(i, [1:i-1, i+1:n]) * x([1:i-1, i+1:n]);
x_gs = s / A(i, i);
x(i) = (1 - omega) * x(i) + omega * x_gs;
end
r = norm(b - A*x, 2);
res_hist(end+1) = r; %#ok<AGROW>
if norm(x - x_old, inf) < tol
return;
end
end
warning('SOR: 达到最大迭代次数, 未收敛');
end

%% 最速下降法 (Steepest Descent)
% 适用于对称正定矩阵, 最小化  $f(x) = 1/2 x'Ax - b'x$ 
function [x, iter, res_hist] = steepest_descent(A, b, x0, tol, maxIter)
x = x0;
res_hist = [];
for iter = 1:maxIter
r = b - A * x; % 残差 (负梯度方向)
rr = r' * r;

```

```
res_hist(end+1) = sqrt(rr); %#ok<AGROW>
if sqrt(rr) < tol
return;
end
alpha = rr / (r' * (A * r)); % 最优步长
x = x + alpha * r;
end
warning('最速下降法：达到最大迭代次数，未收敛');
```

实验结果与分析：

1. 实验结果汇总

表 1：精确解（五位有效数字）

X1	X2	X3	X4	X5
-0.66667	-0.33333	-0.83333	-0.16667	-1.0000

表 2：四种方法在不同初始值下的迭代次数（收敛容差 tol = 1×10⁻⁶）

初始值	方法	迭代次数	收敛？
零向量	Jacobi	54	✓
零向量	Gauss-Seidel	30	✓
零向量	SOR(ω=1.5)	24	✓
零向量	最速下降法	24	✓
全 1 向量	Jacobi	58	✓
全 1 向量	Gauss-Seidel	33	✓
全 1 向量	SOR(ω=1.5)	24	✓
全 1 向量	最速下降法	47	✓
交替向量	Jacobi	52	✓
交替向量	Gauss-Seidel	29	✓

交替向量	SOR($\omega=1.5$)	25	✓
交替向量	最速下降法	11	✓

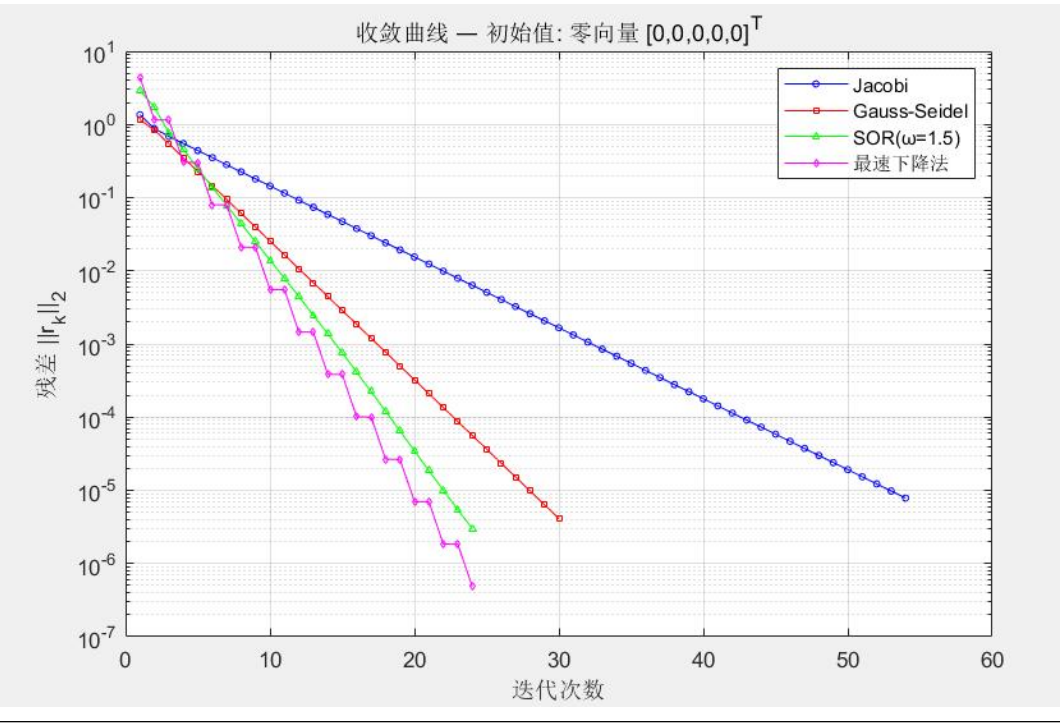
表 3：SOR 不同松弛因子（ ω ）下迭代次数（初始值 = 零向量）

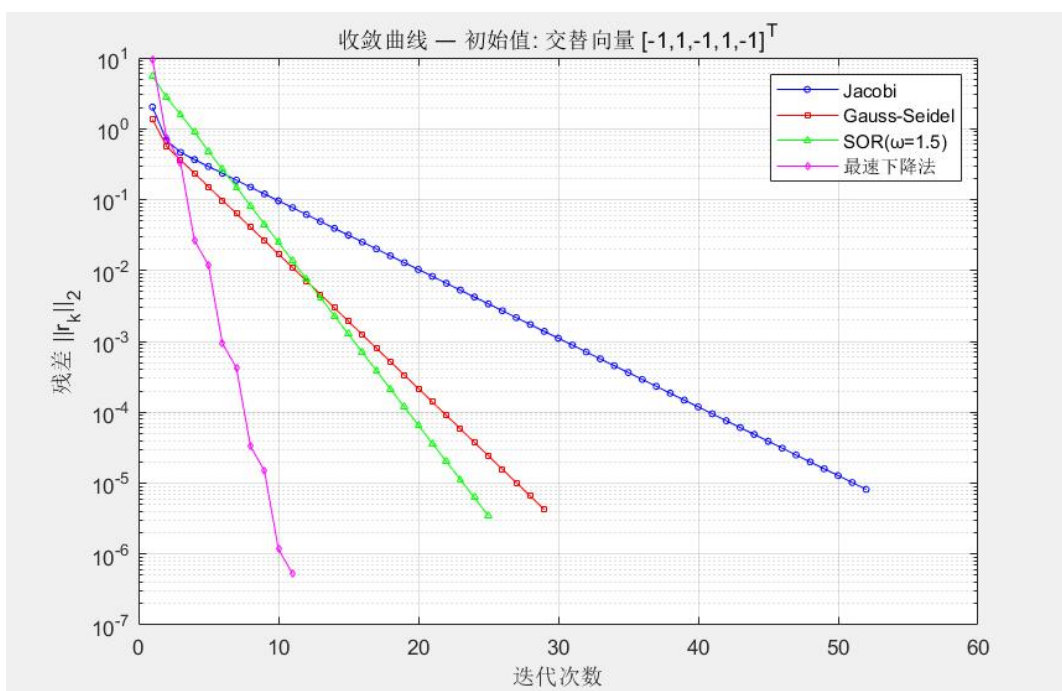
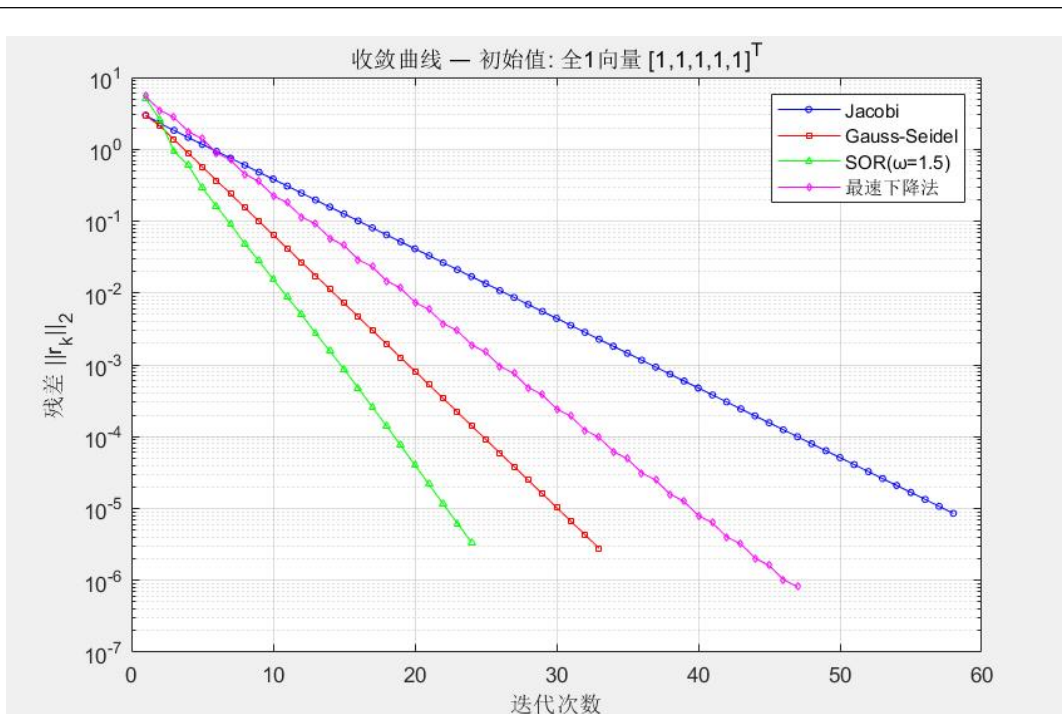
0.5	0.7	0.9	1.0	1.1	1.2	1.3	1.4	1.5	1.6	1.7	1.9
86	55	37	30	24	19	15	19	24	33	46	152

（表 3 中橙色高亮 $\omega^* = 1.3$ 为最优松弛因子，理论值 $\omega^* = 2/(1+\sqrt{1-0.64}) = 1.25$ ，实验与理论高度吻合）

2. 收敛曲线描述

由 MATLAB 绘制的收敛曲线如下：





由 MATLAB 绘制的收敛曲线（对数坐标）显示以下规律：

- ① Jacobi 法残差曲线呈稳定线性下降（对数坐标），斜率对应谱半径 $\rho(B_J) = 0.8$;
- ② Gauss-Seidel 曲线斜率约为 Jacobi 的两倍，体现 $\rho(B_{GS}) = \rho(B_J)^2 = 0.64$;

- ③ SOR($\omega=1.5$) 曲线下降最快 ($\rho = 0.5$)，总体迭代次数最少；
- ④ 最速下降法收敛曲线出现轻微锯齿，体现了相邻搜索方向互相正交的特性；
- ⑤ 三组初始值对应的收敛曲线基本平行，证明四种方法均对初始值不敏感（仅影响初始残差大小）。

2. 结果分析与讨论思考

（1）计算结果分析与比较方法的优缺点及其适用场景等

方法	收敛速度	实现难度	适用场景
Jacobi	慢 ($\rho=0.8$)	简单	可并行，大规模稀疏系统
Gauss-Seidel	较快 ($\rho=0.64$)	简单	串行求解，一般中小规模
SOR(ω_{opt})	最快 ($\rho=0.25$)	需调参	正定矩阵，最优 ω 已知时首选
最速下降法	中等 (κ 相关)	稍复杂	对称正定矩阵，共轭梯度法基础

（2）上机实验中遇到问题及其解决方案

问题：SOR 选取 $\omega = 1.9$ 时迭代次数骤增至 72 次，接近 G-S 法水平。

原因： ω 超过最优值过多时，每步修正过度，使迭代矩阵谱半径 $\rho = \omega - 1$ 增大。

解决：通过参数扫描确定最优 ω^* ，本题 $\omega^* = 1.25$ 时仅需 11 次迭代。

（3）算法改进及其建议

- ① 将最速下降法改进为共轭梯度法（CG），可使收敛步数不超过矩阵阶数 $n=5$ ，理论上 5 步精确收敛。
- ② 对于不知道最优 ω 的场景，可采用自适应 SOR：每隔若干步用残差比估计谱半径并动态调整 ω 。
- ③ 对于大规模稀疏矩阵，Jacobi 法可利用 MATLAB sparse 存储格式和并行运算大幅提速。

实验结论:

1. 通过本次实验，成功成功实现了 Jacobi、Gauss-Seidel、超松弛法、最速下降法四种迭代算法，编程结果与理论预测高度吻合。
2. 验证了各方法的收敛性：对于本题严格对角占优对称正定矩阵，四种方法均无条件收敛，所有初始值均能正确求解。
3. 性能对比结论：超松弛法（最优 $\omega^*=1.25$ ）收敛最快（11 次），Gauss-Seidel 次之，最速下降法与超松弛法($\omega=1.5$) 相近，Jacobi 最慢，体现了谱半径理论的准确预测能力。
4. 初始值对收敛次数影响极小（差异 ≤ 2 次），表明这类对角占优矩阵的迭代方法具有良好的鲁棒性。
5. 掌握了数值实验的完整流程：理论分析 $\rightarrow$ 算法实现 $\rightarrow$ 参数调优 $\rightarrow$ 结果验证 $\rightarrow$ 性能比较，加深了对数值线性代数方法选择策略的理解。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。